VigiaVia RPA

Automação Inteligente de Ocorrências Viárias

FIAP Global Solution 2026.2 · CitizenTech · Robotic Process Automation

Integrante	RM
Deivison Pertel	550803
Eduardo Akira Murata	98713
Wesley Souza de Oliveira	97874

Relatório técnico — versão inicial para revisão da equipe

1. Contexto e problema

O VigiaVia é uma proposta CitizenTech de zeladoria inteligente de vias públicas. O problema abordado são buracos e danos no pavimento cuja identificação pode depender de chamados manuais, lentos, imprecisos e duplicados. O RPA automatiza a triagem operacional dos registros antes de sua disponibilização a um componente de gestão.

2. Objetivo e benefício

Automatizar o recebimento em lote, validar informações, identificar possíveis duplicidades geográficas, classificar prioridades e gerar saídas estruturadas com rastreabilidade. O benefício esperado é reduzir conferências repetitivas e melhorar a organização dos chamados, sem substituir a validação humana.

3. Escopo e arquitetura

A solução é independente para execução local e utiliza CSV de entrada. O fluxo contém leitor, validador, deduplicação geográfica pelo método Haversine, regra de prioridade, exportador CSV/JSON e logging. Não depende de Docker, Azure ou da API do MVP.

4. Mapeamento do processo (PDD)

Gatilho: operador inicia o comando Python. Entrada: CSV com id, data, latitude, longitude, gravidade e descrição. O robô verifica estrutura, percorre cada linha, valida os campos, compara coordenadas com registros válidos anteriores, classifica prioridade e exporta os resultados. Saídas: CSV completo, JSON de integração e log.

5. Regras de negócio e decisões

Campos obrigatórios ausentes, data inválida, coordenadas não numéricas ou fora do intervalo e gravidade desconhecida resultam em INVALIDO. Gravidade alta, média ou baixa é mapeada para prioridade ALTA, MEDIA ou BAIXA. Uma distância de até 20 metros de registro válido anterior gera POSSIVEL_DUPLICIDADE. O raio de 20 metros é apenas um parâmetro demonstrativo configurável, não uma regra oficial do MVP.

6. Repetição e processamento em lote

Um laço for percorre todas as ocorrências do CSV. Uma falha isolada em uma linha é registrada e não impede o processamento das linhas seguintes. Os itens aprovados são acumulados para o arquivo JSON de integração.

7. Exceções e resiliência

Arquivo ausente, CSV vazio, cabeçalho incorreto e falhas de leitura/escrita geram mensagem e log. Dados inválidos são isolados por registro. Possíveis duplicidades são sinalizadas sem exclusão automática. Falha de gravação impede que a execução seja apresentada como concluída com sucesso.

8. Logs e auditoria

Os logs registram horário, identificador de execução, início/fim, identificadores técnicos, status, prioridade, contagens e tipos de erro. O log não reproduz descrição completa nem dados pessoais. O CSV de resultado registra o status e o motivo de cada linha.

9. Integração com o MVP

A saída JSON versionada é gravada em data/mvp_inbox/ e representa a integração por arquivo estruturado com outro componente CitizenTech. Este pacote comprova a produção do arquivo, mas não chama a API real nem grava no banco do MVP. Para comprovar integração ponta a ponta, a equipe deve demonstrar um consumidor lendo esse arquivo e registrar evidência real.

10. Testes e validação

O pacote inclui testes unitários para validação, prioridade, cálculo de distância e processamento em lote, executáveis com python -m unittest discover -s tests -v. O plano de testes também prevê arquivo ausente, estrutura incorreta, dados inválidos, duplicidade e lote misto.

11. Segurança e LGPD

O conjunto de demonstração é sintético e não contém nome, telefone, e-mail, placa ou imagem. Não inserir dados pessoais no CSV. Coordenadas podem ser sensíveis dependendo do contexto e devem ter acesso restrito e retenção definida pelo operador. Casos de duplicidade permanecem sujeitos à revisão humana.

12. Tecnologias

Python 3.10 ou superior, biblioteca padrão (csv, json, logging, datetime, pathlib, unittest) e cálculo geográfico Haversine. Não são necessárias dependências externas para a execução principal.

13. Repositório, vídeo e evidências

Repositório GitHub: inserir URL pública após publicação. Vídeo: inserir URL após gravação. Inserir capturas reais de execução, testes, CSV e JSON em assets/evidencias/. O vídeo deve ter até três minutos e demonstrar problema, execução, decisão, integração e tratamento de falhas.

Fluxo BPMN simplificado

```
INÍCIO → Operador inicia RPA → Ler CSV → [Arquivo/cabeçalho válidos?]
Não → Log de falha → FIM COM ERRO
Sim → Para cada ocorrência:
  Validar dados → [Válido?] Não → INVALIDO
  Sim → Calcular distância → [Até 20 m de item aceito?]
    Sim → POSSIVEL_DUPLICIDADE
    Não → Classificar prioridade → PROCESSADO / adicionar ao JSON
Após o lote → Gerar CSV + JSON + resumo → Registrar totais → FIM
```

Prazo informado no desafio: 19/11/2026 às 23h55 (entrega via Teams).

Complemento — Integração demonstrativa por arquivo

Além da geração do JSON estruturado, o pacote inclui um segundo componente local: **src.mvp_consumer**. Ele lê o arquivo JSON produzido pelo RPA e registra as ocorrências em uma base SQLite local, simulando o consumo por outro componente CitizenTech.

Comando (substituir pelo nome real do JSON exibido ao final da execução):

```
python -m src.mvp_consumer data/mvp_inbox/ocorrencias_<execucao>.json
```

A tabela SQLite utiliza occurrence_id como chave primária. Assim, reprocessar o mesmo arquivo não insere novamente a mesma ocorrência. O teste automatizado valida esse comportamento de idempotência.

Limite importante: esta é uma integração local demonstrativa por arquivo estruturado entre dois componentes locais. Não representa conexão com a API nem com o banco de produção do MVP VigiaVia. Para integração real, o consumidor deverá ser adaptado ao contrato efetivo do MVP e a equipe deverá registrar a evidência de consumo no sistema real.

Execução de validação do pacote: 7 registros no CSV de demonstração; 4 processados; 2 inválidos; 1 possível duplicidade; 0 erros. O consumidor inseriu 4 novos registros no SQLite local. Os 12 testes automatizados passaram nesta execução.